In [1]:

```
import pandas as pd
df = pd.read_csv('C:/Kabir/College Work/Semester 6/CSD 330/Project/Data Sets/detecting-insults-in-social-commentary/train.csv', encoding='ISO-8859-1');
```

In [2]:

```
#Import for regular expressions
import re
#Import for tokenisation, lemmatisation, etc.
import nltk
```

In [3]:

```
nltk.download('punkt')
from nltk.tokenize import word_tokenize as wt
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\kabir\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

In [4]:

```
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\kabir\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [5]:

```
from nltk.stem.porter import PorterStemmer
stemmer = PorterStemmer()
```

In [6]:

```
#View dataset schema
df.head()
```

Out[6]:

	Insult	Date	Comment
0	1	20120618192155Z	"You fuck your dad."
1	0	20120528192215Z	"i really don't understand your point.\xa0 It ...
2	0	NaN	"A\\xc2\\xa0majority of Canadians can and has ...
3	0	NaN	"listen if you dont wanna get married to a man...
4	0	20120619094753Z	"C\xe1c b\u1ea1n xu\u1ed1ng \u0111\u01b0\u1edd...

In [7]:

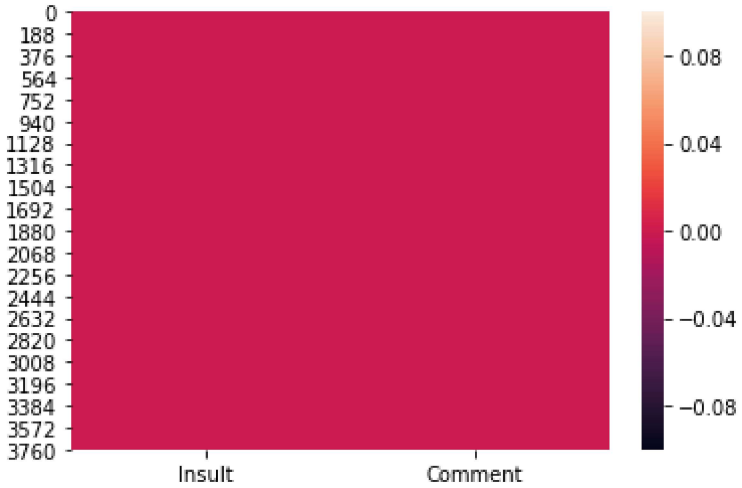
```
#Remove irrelevant features
df = df.drop('Date',axis=1)
```

In [56]:

```
#Check for null values
import seaborn as sb
import matplotlib
sb.heatmap(df.isnull())
```

Out[56]:

```
<matplotlib.axes._subplots.AxesSubplot at 0x2339723cdc8>
```



In [9]:

```
comments = []
for i in range(df.shape[0]):
    comment = df.iloc[i, 1]
    #Regular expression for words with slashes
    slash_all = r'\s*(?:[\w_]*[/\\](?:[\w_]*[/\\])*[\w_]*)'
    #Remove words with slashes
    comment = re.sub(slash_all,' ',comment)
    #Remove special characters except '
    comment = re.sub('[^\'A-Za-z]',' ',comment)
    #Remove extra whitespace
    comment = comment.strip()
    #Change to lower case
    comment = comment.lower()
    #Tokenise
    tokenised_comment = wt(comment)
    #Remove stop words
    comments_processed = []
    for word in tokenised_comment:
        if word not in set(stopwords.words('english')):
            comments_processed.append(stemmer.stem(word))
    comment_text = " ".join(comments_processed)
    comments.append(comment_text)
```

In [10]:

```
df2 = pd.read_csv('C:/Kabir/College Work/Semester 6/CSD 330/Project/Data Sets/detecting
-insults-in-social-commentary/test_with_solutions.csv', encoding='ISO-8859-1');
```

In [11]:

```
df2.head()
```

Out[11]:

	Insult	Date	Comment	Usage
0	0	NaN	"THE DRUDGE REPORT\\n\\n\\n\\nYou won't see th...	PublicTest
1	0	20120618222256Z	"@ian21\xa0"Roger Clemens is the fucking man, ...	PublicTest
2	1	20120618213617Z	"Agree with Alan you are an extremest idiot. ...	PublicTest
3	0	NaN	"Really?\\n\\nI see Marc Lamont Hill on variou...	PrivateTest
4	0	20120620003825Z	"Really suck isn't the word, when many of our ...	PrivateTest

In [12]:

```
cols_to_drop = ['Date','Usage']
df2 = df2.drop(cols_to_drop,axis=1)
```

In [13]:

```
df2.head()
```

Out[13]:

	Insult	Comment
0	0	"THE DRUDGE REPORT\\n\\n\\n\\nYou won't see th...
1	0	"@ian21\xa0"Roger Clemens is the fucking man, ...
2	1	"Agree with Alan you are an extremest idiot. ...
3	0	"Really?\\n\\nI see Marc Lamont Hill on variou...
4	0	"Really suck isn't the word, when many of our ...

In [14]:

```
#Check for null values
sb.heatmap(df2.isnull())
```

Out[14]:

```
<matplotlib.axes._subplots.AxesSubplot at 0x233e762c108>
```

In [15]:

```
comments_test = []
for i in range(df2.shape[0]):
    comment = df2.iloc[i, 1]
    #Regular expression for words with slashes
    slash_all = r'\s*(?:[\w_]*[/\\](?:[\w_]*[/\\])*[\w_]*)'
    #Remove words with slashes
    comment = re.sub(slash_all,' ',comment)
    #Remove special characters except '
    comment = re.sub('[^\'A-Za-z]',' ',comment)
    #Remove extra whitespace
    comment = comment.strip()
    #Change to lower case
    comment = comment.lower()
    #Tokenise
    tokenised_comment = wt(comment)
    #Remove stop words
    comments_processed = []
    for word in tokenised_comment:
        if word not in set(stopwords.words('english')):
            comments_processed.append(stemmer.stem(word))
    comment_text = " ".join(comments_processed)
    comments_test.append(comment_text)
```

In [16]:

```
# Create feature matrix
from sklearn.feature_extraction.text import CountVectorizer
matrix = CountVectorizer()
```

In [17]:

```
X_train = matrix.fit_transform(comments).toarray()
y_train = df.iloc[:, 0]
X_test = matrix.transform(comments_test).toarray()
y_test = df2.iloc[:, 0]
```

In [19]:

```
#Naive Bayes
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# predict class
y_pred = classifier.predict(X_test)

# Confusion matrix
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
```

In [20]:

```
print(accuracy)
```

```
0.6422364941443143
```

In [22]:

```
print(cm)
```

```
[[1296  658]
 [ 289  404]]
```

In [23]:

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.82      0.66      0.73      1954
           1       0.38      0.58      0.46       693

    accuracy                           0.64      2647
   macro avg       0.60      0.62      0.60      2647
weighted avg       0.70      0.64      0.66      2647
```

In [25]:

```
# Using simple Decision Tree classifier
from sklearn.tree import DecisionTreeClassifier
```

In [42]:

```
classifier = DecisionTreeClassifier(max_depth=5)
classifier.fit(X_train, y_train)

# predict class
y_pred = classifier.predict(X_test)

# Confusion matrix
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
```

In [43]:

```
print(accuracy)
```

```
0.7842840952021156
```

In [44]:

```
print(cm)
```

```
[[1877   77]
 [ 494  199]]
```

In [45]:

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.79      0.96      0.87      1954
           1       0.72      0.29      0.41       693

    accuracy                           0.78      2647
   macro avg       0.76      0.62      0.64      2647
weighted avg       0.77      0.78      0.75      2647
```

In [49]:

```
#SVM
from sklearn.svm import SVC
```

In [50]:

```
classifier = SVC()
classifier.fit(X_train, y_train)

# predict class
y_pred = classifier.predict(X_test)

# Confusion matrix
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)

accuracy = accuracy_score(y_test, y_pred)
```

In [51]:

```
print(accuracy)
```

```
0.8111069134869664
```

In [52]:

```
print(cm)
```

```
[[1896   58]
 [ 442  251]]
```

In [53]:

```
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.81      0.97      0.88      1954
           1       0.81      0.36      0.50       693

    accuracy                           0.81      2647
   macro avg       0.81      0.67      0.69      2647
weighted avg       0.81      0.81      0.78      2647
```